

General Overview, Practical Recommendations & GDB

Class 1 – Beginner-Friendly and Advanced C Programming

Jakub Piela

AGH University of Krakow · Faculty of Physics and Applied Computer Science
“KERNEL” CS Student Research Club · kernel.fis.agh.edu.pl

Academic Year 2025/2026

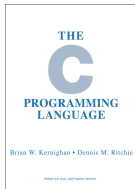


Educational material prepared for the “KERNEL” CS Student Research Club, AGH University of Krakow. Provided *as is*, for learning purposes only; the author assumes no responsibility for its use or for any errors herein. All third-party names, trademarks, and book covers belong to their respective owners.

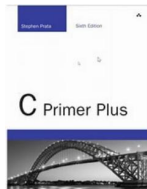
Outline

- 1 Recommended Reading & Resources
- 2 C23 Essentials & Recommendations
- 3 Developer Tooling
- 4 Debugging with GDB

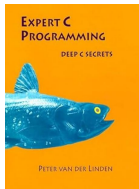
Recommended books on C



C Prog. Lang.
Kernighan & Ritchie



C Primer Plus
S. Prata



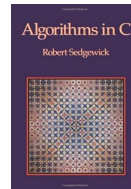
Expert C
van der Linden



Modern C
J. Gustedt



Inside C
Di Cataldo



Algorithms in C
R. Sedgewick

Note

Modern C (Gustedt, Manning 2025) is the backbone for the C23 material in these lectures and is cited throughout.

Book covers are the property of their respective authors and publishers, reproduced here for educational reference only — the author claims no ownership of them. If you hold the rights and object, contact us and they will be removed.

Websites & tooling references

- [c-language.org](https://en.cppreference.com/) — C language reference
- [Jens Gustedt's blog](#) — author of *Modern C*
- [Compiler Explorer \(godbolt.org\)](https://godbolt.org/) — C → assembly, live, many compilers
- [IOCCC](#) — International Obfuscated C Code Contest

Tip

Compiler note. GCC 14.2.0 (ideally 15.2.0) is strongly recommended — older versions may not support all C23 features used here.

Selected YouTube channels

- [Tsoding](#) — live systems coding
- [cacharle](#) — advanced C
- [Magicalbat](#) — low-level C
- [Jacob Sorber](#) — embedded & C
- [Eskil Steenberg](#) — C philosophy
- [CppCon](#) — conference talks
- [Low Level TV](#) — reverse eng. & security
- [LaurieWired](#) — malware analysis
- [Daniel Hirsch](#) — C projects
- [Write Your Own OS](#) — OS in C

The main function: valid prototypes

`main` is the entry point — its prototype is fixed by the standard, yet implemented by you. Two forms are portable everywhere:

```
1 int main(void);  
2 int main(int argc, char* argv[argc+1]);
```

Recommendation

Do not rely on platform-specific prototypes of `main` (e.g. `void` return, a third `envp` parameter). Stick to the two official forms for portable code.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §7.2.

The main function: return values

Only two return values are guaranteed portable, both from `<stdlib.h>`:

Value	Meaning
EXIT_SUCCESS	Program completed successfully
EXIT_FAILURE	Program encountered an error

```
1 #include <stdlib.h>
2 int main(void) {
3     return EXIT_SUCCESS;
4 }
```

Recommendation

Always return EXIT_SUCCESS/EXIT_FAILURE; never bare 0 or 1.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §7.2, Takeaway 7.2 #1.

The implicit return of main

Any void function may omit `return`. `main` is special because omitting it still *returns a value*: reaching the end of `main` behaves as `return EXIT_SUCCESS`.

```
1 // Valid but not recommended -- implicit return
2 int main(void) { printf("Hello, World!\n"); }
3 // Recommended -- always be explicit
4 int main(void) {
5     printf("Hello, World!\n");
6     return EXIT_SUCCESS;
7 }
```

From *Modern C*

Reaching the end of `main` $\equiv$ `return EXIT_SUCCESS` (Takeaway 7.2 #2); `argv[argc]` is null (Takeaway 7.2 #7).^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §7.2.

Null pointers: prefer `nullptr` over `NULL`

The cleanest null in modern C is `nullptr` (C23):

```
1 | int* ptr = nullptr;
```

`NULL`'s underlying type is *not* standardized — it may expand to `0`, `0L`, or `(void*)0`. This bites hardest in variadic calls, where the type cannot be inferred.

Recommendation

Never use `NULL`: use plain `0` in integer contexts, or `nullptr` for a null pointer.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §6.2, Takeaway 6.2 #3.

Compiling with C23 & inspecting output

```
gcc -std=c2x source.c -o output      # enable C23
gcc -std=c23 source.c -o output      # newer GCC
gcc -std=c2x -E source.c | less      # preprocessor: macro expansions
gcc -std=c2x -S -O0 source.c          # generated assembly (-O0 reads easiest)
gcc -std=c2x -g source.c -o output    # debugging symbols for GDB
```

Tip

Recommended development command:

```
gcc -std=c2x -g -Wall -Wextra -O0 source.c -o output
```

Compiler flag reference

Flag	Meaning
-std=c2x	Enable the C23 standard
-g	Debugging symbols (required for GDB)
-Wall -Wextra	Common + extra warnings
-Wpedantic	Enforce strict ISO C compliance
-Werror	Treat warnings as errors
-O0 / -O2	No optimization (debug) / production default
-O3	Aggressive; may grow the binary
-Ofast	-O3 + non-IEEE math (breaks compliance)
-S / -E	Emit assembly / run preprocessor only

Static analysis: cflow & cscope

cflow — call-graph generator; trace which function calls which:

```
cflow source.c           # call graph
cflow -r source.c        # reverse: who calls this?
```

cscope — interactive code browser for large projects:

```
cscope -Rb               # build database
cscope -d                 # launch browser
# Ctrl+| s  symbol      Ctrl+| g  definition  Ctrl+| c  callers
```

Memory errors: Valgrind

Detects leaks, invalid accesses, and use of uninitialized memory.

```
gcc -std=c2x -g source.c -o output      # -g adds source line info
valgrind --leak-check=full ./output

valgrind --leak-check=full --show-leak-kinds=all \
        --track-origins=yes --verbose ./output
```

Recommendation

Valgrind finds leaks and errors even without `-g`; adding `-g` just lets its reports point at the exact source line.

Decoding declarations: cdecl

C declarations get cryptic; cdecl translates both ways (also online at cdecl.org).

```
cdecl explain "int (*fp)(int, int)"  
# declare fp as pointer to function (int, int) returning int  
  
cdecl explain "char* (*argv)[10]"  
# declare argv as pointer to array 10 of pointer to char
```

Note

Expert C Programming (van der Linden) presents a minimal cdecl as a finite state machine — intentionally without error checking.

Inspecting binaries: Linux one-liners

```
hexdump -C output      # raw bytes (hex + ASCII)
strings output          # human-readable strings in a binary
strings -t x output     # with file offsets
nm -n output            # symbols, sorted by address
nm -C output            # demangle (C++ symbols)
objdump -d output       # disassemble all sections
objdump -S output       # disassemble + source (needs -g)
strace ./output         # trace system calls
strace -c ./output      # summarise syscalls
```

GDB is the standard C debugger on Linux: run under control, pause anywhere, inspect and modify memory/variables, trace calls, analyse crashes post-mortem. **DDD** is a graphical front-end for GDB.

Recommendation

Always compile with `-g` before debugging.
`gcc -std=c2x -g -Wall -Wextra source.c -o output`

Recommended videos: [GDB — practical walkthrough](#) • [GDB — extended deep-dive](#).

Starting GDB & passing arguments

```
gdb ./output                # start
gdb -ex run ./output        # start and run immediately

(gdb) set args arg1 arg2 arg3  # set argc/argv, then:
(gdb) run

gdb --args ./output arg1 arg2  # or pass on the command line
```

TUI mode — the single most useful feature

Splits the terminal into source / assembly / console panes.

```
(gdb) tui enable          (gdb) layout src      # source only (key one)
Ctrl+x a  (toggle TUI)  (gdb) layout asm      # assembly
                        (gdb) layout split  # source + assembly
                        (gdb) layout regs   # add registers
```

Tip

`layout src` shows your code with a live arrow on the current line. Typical start: `layout src` → `break main` → `run`.

Navigating inside TUI mode

```
Ctrl+p / Ctrl+n      # previous / next command (arrows scroll source!)
Ctrl+b / Ctrl+f      # move cursor left / right
PageUp / PageDown    # scroll the source window
Ctrl+x o             # switch focus between panes
Ctrl+x 1 / Ctrl+x 2  # single / two-window layout
Ctrl+l               # redraw (fixes a garbled display)
```

Note

In TUI mode the arrow keys scroll the source pane — use Ctrl+p/Ctrl+n to move through command history.

Breakpoints & watchpoints

```
(gdb) break main                # at a function
(gdb) break source.c:42         # at a line
(gdb) info breakpoints          # list
(gdb) delete 1 / disable 1 / enable 1

# Watchpoints -- pause when a variable is accessed
(gdb) watch variable            # on write
(gdb) rwatch variable          # on read
(gdb) awatch variable           # on read or write
```

Stepping & execution control

```
(gdb) run / continue      # start / continue to next breakpoint
(gdb) next                # step over (don't enter calls)
(gdb) step                # step into calls
(gdb) finish              # run until current function returns
(gdb) until 55            # run until line 55
(gdb) jump 60             # jump execution to line 60
(gdb) kill / quit
```

Inspecting & modifying state

```
(gdb) print variable / print arr[2] / print *ptr / print &variable
(gdb) display variable           # auto-print at each step
(gdb) info locals / info args / info registers
(gdb) ptype my_variable          # full type
(gdb) whatis my_variable         # type name
(gdb) print sizeof(my_struct)    # evaluate any C expression
(gdb) x/10x 0x7fffffff000        # examine 10 hex words at address

(gdb) set var result = 42        # modify a variable at runtime
(gdb) set var arr[0] = 999
```

Call stack & backtraces

```
(gdb) backtrace           # full call stack (bt)
(gdb) backtrace full      # + local variables
(gdb) frame 0 / frame 2   # select a frame
(gdb) up / down           # move one frame
(gdb) info frame          # details of current frame
```

Hook functions — automate inspection

Commands named `hook-<cmd>` run automatically *before* a command (`hookpost-<cmd>` runs after):

```
(gdb) define hook-next
> info locals
> backtrace 3
> end

(gdb) define hookpost-next
> display i
> end
```


Calling functions while paused

GDB can call any function in your program — including your own debug helpers — without recompiling:

```
(gdb) call printf("hello from GDB\n")
(gdb) call debug_dump_state(&state)
(gdb) print my_function(42)           # call + show return value
```

Recommendation

Write `debug_*` helpers from the start; guard them with `#ifdef DEBUG` so they cost nothing in release, and call them live from GDB.

Checkpoints, breakpoint files & split symbols

```
(gdb) checkpoint / info checkpoints / restart 1    # snapshot state

(gdb) save breakpoints bp.txt          # persist breakpoints
(gdb) source bp.txt                    # reload them

# Strip symbols for release, keep them for later debugging
objcopy --only-keep-debug main gdbtest.dbg
objcopy --strip-debug main
gdb ./main -s gdbtest.dbg
```

Core dumps — post-mortem debugging

A core dump is a snapshot of memory at the moment of a crash.

```
ulimit -c unlimited           # enable (off by default)
coredumpctl -o coredump dump ./main # collect via systemd
gdb ./main -c coredump        # open in GDB
(gdb) backtrace ; frame 0 ; info locals
```

Note

Demo files ship with the notes: `coredump_demo.c` (triggers a segfault) and `gdb_demo.c` (breakpoints, watchpoints, stepping, `argc/argv`, pointer arithmetic, recursive backtraces).

Attaching & remote debugging

```
# Attach to a running process  
ps aux | grep program_name  
(gdb) attach PID  
(gdb) detach # detach without killing it  
  
# Remote debugging with gdbserver  
gdbserver localhost:1234 ./program # on the target  
(gdb) target remote localhost:1234 # on the host
```

DDD (graphical front-end): `sudo apt install ddd` then `ddd ./output.`

GDB quick reference

TUI

layout src — source pane (key)
layout split — source + asm
focus src/cmd — switch focus
Ctrl+x a — toggle TUI

Breakpoints

break main / break f.c:42
watch var
save breakpoints f.txt

Execution

run / continue
next / step / finish
checkpoint / restart N

Inspection

print var / display var
info locals / info args
ptype var
backtrace / frame N
set var x = 42
call my_function(args)

Advanced

target core coredump
symbol-file file.dbg
attach PID / detach
define hook-step

Compile with `-g -Wall -Wextra`, prefer `nullptr`
and `EXIT_SUCCESS`, and live in layout src.

- J. Gustedt, *Modern C*, 3rd ed., Manning, 2025. ISBN 978-1633437777 — §6.2 (pointers, `nullptr`), §7.2 (`main`). [hal-02383654v2](#)
- GDB manual and the recommended GDB video walkthroughs (links on the GDB overview slide).
- Tooling docs: `cflow`, `cscope`, `valgrind`, `cdecl` ([cdecl.org](#)), [Compiler Explorer](#).

Based on the KERNEL class notes by Jakub Piela. © 2026.